Hybrid Search

keywords for precision, vectors for meaning

Watch the 4-min video

In one line: run **BM25** and **vector** search in parallel and merge the two ranked lists, usually with **Reciprocal Rank Fusion** (score = Σ 1/(k + rank), k ≈ 60), which needs no score normalisation; optionally re-rank the top results with a **cross-encoder**. Each method covers the other's blind spot.

1 Each search fails where the other shines

A shopper searches for the exact product code **XB-2017**; vector search returns semantically similar products (XB-2019, XB-2018) but not that one. Another types **something to keep my coffee hot on the go**; keyword search finds nothing, because no product title uses those words. A production search system has to handle both kinds of query, often without knowing in advance which one it got.

Keyword (BM25) wins
codes, SKUs, part numbers, error messages
names, rare or brand-new jargon
short precise queries; legal/medical terms
explainable: you can see what matched

Vector wins
long natural-language questions
paraphrases, synonyms, vague intent
cross-language queries (multilingual models)
no exact word overlap with the answer

Figure 1. Strengths of each retriever. Hybrid search is the default answer when queries mix both kinds.

Why vectors miss codes: an embedding compresses a whole text into a few hundred numbers that capture meaning; 'XB-2017' and 'XB-2019' differ by one character and mean almost the same thing to the model, so their vectors nearly coincide. BM25 (lesson 17) treats `xb-2017` (kept intact by a keyword or code-aware analyzer) as a rare term with a high IDF, so the exact product wins by a mile. Conversely, BM25 can't connect 'coffee hot' to 'insulated travel mug' unless someone wrote that synonym.

2 Architecture

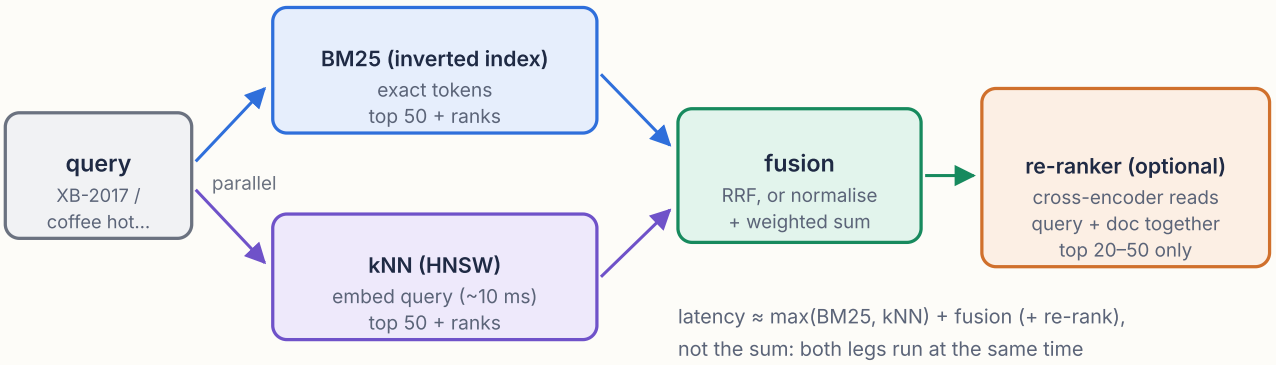


Figure 2. Hybrid query: two retrievers in parallel, a fusion step, and an optional re-ranker on the short list.

Both legs retrieve a candidate window (e.g. top 50–100 each), with the same filters applied to both (tenant, language, stock). Many engines run this as one request: Elasticsearch's `rrf` retriever,

OpenSearch's `hybrid` query with a normalisation search pipeline, and built-in hybrid modes in vector databases such as Weaviate, Qdrant and Vespa. A further option on the keyword side is **learned sparse retrieval** (SPLADE, Elastic's ELSER): a model expands text into weighted terms, adding some semantic matching while keeping an inverted index.

3 Why you can't just add the scores

BM25 scores are unbounded and depend on the query (12.5 for one query, 41 for another). Cosine similarity sits in a narrow band (e.g. 0.79–0.83 for the top 10). Adding them lets BM25 dominate completely. **Weighted (convex) fusion** first normalises each list, then mixes, typically with min–max normalisation norm(x) = (x − min)/(max − min) per result list:

$$s = \alpha \cdot \mathrm{norm}(s_{vec}) + (1 - \alpha) \cdot \mathrm{norm}(s_{BM25})$$

With α = 0.7 the mix is 70% semantic, 30% keyword. It can beat RRF when tuned on a judgement set (lesson 17), because it keeps information about *how much* better one result is. But min–max normalisation is fragile: it stretches the tiny cosine differences 0.79–0.83 across the full 0–1 range, and one outlier score squashes everything else. α also depends on the query type, so it needs tuning and re-tuning when models change.

4 Reciprocal Rank Fusion

$$\mathrm{RRF}(d) = \Sigma_{lists}\ 1 \,/\, (k + \mathrm{rank}(d)),\ k = 60$$

RRF (Cormack, Clarke and Büttcher, 2009) ignores raw scores and uses only ranks, so the two lists never need to be comparable; a document missing from a list simply adds 0 for it. k damps the advantage of the very top positions: with k = 60, rank 1 is worth 0.0164 and rank 10 still 0.0143, so appearing in **both** lists matters more than being first in one. Being ranked 1st and 5th (1/61 + 1/65 = 0.0318) is almost the same as 3rd and 3rd (2/63 = 0.0317). There is nothing to train and it is robust when you change models, which is why it is the common default; if you have judged queries, test whether tuned weighted fusion does better on your data.

Product (query 'XB-2017')	BM25 rank	vector rank	RRF score	final
XB-2017 travel mug	1	3	1/61 + 1/63 = 0.0323	**1**
XB-2017 replacement lid	2	6	1/62 + 1/66 = 0.0313	2
XB-2019 travel mug	—	1	1/61 = 0.0164	3
XB-2018 travel mug	—	2	1/62 = 0.0161	4

5 Re-ranking the top results

Both retrievers are **bi-encoders** or term matchers: documents are encoded independently of the query, which is what makes them indexable. A **cross-encoder** re-ranker feeds the query and one candidate through a transformer *together*, so it sees how every query word relates to every document word. It is far more accurate and far slower (a model call per candidate), so it runs only on the fused top 20–50, often adding tens of milliseconds on a GPU or via a hosted re-rank API. This retrieve → fuse → re-rank funnel is also the standard retrieval stack for RAG.

WORKED EXAMPLE: EXACT CODES AND FUZZY INTENT IN ONE SEARCH BOX

Every query runs BM25 (on a `name` text field plus a `sku` keyword field) and kNN (on 768-dim product embeddings) in parallel, each returning its top 50 with the same in-stock filter. RRF with k = 60 merges them and the page shows the top 20. **'XB-2017'**: BM25 ranks the exact product 1st and its lid 2nd; the vector side ranks XB-2019 first, but the exact product, present in both lists, wins (table above). **'something to keep my coffee hot on the go'**: BM25 matches loosely on 'coffee' (coffee beans, coffee grinders), while the vector side ranks insulated travel mugs and thermos flasks at the top; mugs appear in both lists at moderate ranks and rise to the top after fusion. Latency is the slower of the two legs plus a millisecond of fusion.

TRADE-OFFS

- **RRF**: no tuning, robust; discards score magnitude, so a clear BM25 winner gets no extra credit.
- **Weighted fusion**: can be more accurate when tuned; fragile normalisation, α per query type.
- **Cross-encoder re-ranking**: big quality gains; costs latency and GPU or API spend per query.
- **Two indexes**: more storage and two pipelines to keep in sync (lesson 19).

COMMON MISTAKES

- Adding raw BM25 and cosine scores.
- Applying filters to only one leg, so fused results include forbidden or out-of-stock items.
- Analysing product codes with the standard analyzer, which splits 'XB-2017' into 'xb' and '2017'.
- Choosing hybrid weights by gut feel instead of judged queries and A/B tests.

6 Key terms

Hybrid search: keyword and vector retrieval, fused

Weighted fusion: normalise scores, mix with weight α

Cross-encoder: re-ranker reading query + doc together

RRF: Σ 1/(k + rank) across lists, k ≈ 60

Min–max normalisation: rescale a list's scores to 0–1

Learned sparse: model-weighted terms in an inverted index

7 Interview questions

Q1. Why combine keyword and vector search? BASIC

Keyword search is precise for exact terms like codes and names, vector search understands meaning and paraphrases; together they cover each other's blind spots.

Q2. Explain RRF. BASIC

Each document scores the sum of 1/(k + rank) over the result lists it appears in (k ≈ 60), so rankings merge without comparing incompatible scores.

Q3. Users of an internal docs search complain that error codes don't find the right page after you moved to embeddings. Fix? SCENARIO

Add a BM25 leg with a keyword or code-preserving analyzer and fuse with RRF; exact tokens like error codes are where vectors are weakest.

Q4. You have 500 judged queries. RRF or weighted fusion? SCENARIO

Evaluate both offline with NDCG; weighted fusion may win when tuned, but keep RRF if the gain is small, because it's robust to model and data changes.

Q5. Hybrid quality is good but you want more. Next step? SCENARIO

Add a cross-encoder re-ranker on the fused top 20–50, measure NDCG and latency, and cache or batch re-rank calls.

Q6. Why is k = 60 and not 1? TRICKY

With k = 1, rank 1 (0.5) dwarfs rank 2 (0.33), so one list's top hit dominates; k = 60 flattens the curve so agreement between lists matters most. 60 is the value from the original paper and works well broadly.

Q7. Doesn't hybrid search double latency? TRICKY

No: the legs run in parallel, so latency is roughly the slower leg plus fusion; cost (CPU, storage) is what roughly doubles.

Next: SD-P-012 · Search Scaling backendroadmap.com